

Q1 Program based on Classes and Objects - Initializaing Data member using Objects directly

Write a Java program to create a class named Student with the following data members:

int rollNo
String name
double marks

Create a method named displayDetails() to display the student's details.

In the main() method:

Create a Student object.

Read the values from the user.

Initialize the data members directly using the object (do not use a constructor or setter methods).

Invoke the displayDetails() method using the object.

Input Format

The first line contains an integer representing the roll number.

The second line contains the student's name.

The third line contains a decimal number representing the student's marks.

Output Format

Print the details in the following format:

Roll No: <rollNo>

Name: <name>

Marks: <marks>

Your Code

```
1 import java.util.Scanner;
2 public class Main{
3     public static void main(String[]args){
4         Scanner s1=new Scanner(System.in);
5         Student s=new Student();
6
7         s.rollNo=s1.nextInt();
8         s1.nextLine();
9         s.name=s1.nextLine();
10        s.marks=s1.nextDouble();
11
12        s.displayDetails();
13    }
14 }
15 class Student{
16     public int rollNo;
17     public String name;
18     public double marks;
19
20     public void displayDetails(){
21         System.out.println("Roll No: "+rollNo);
22         System.out.println("Name: "+name);
23         System.out.println("Marks: "+marks);
24
25     }
26 }
```

Input

```
21
Mohamed Imraan
85.75
```

Output

```
Roll No: 21
Name: Mohamed Imraan
Marks: 85.75
```

Visible Test Cases

Test Case 1	✓
Test Case 2	✓
Test Case 3	✓

10.00 / 10 marks

Q2 Classes and Objects-Initialize Object Data Using Setter Methods

Write a Java program to create a class named Student with the following private data members:

```
int rollNo
String name
double marks
```

Create the following setter methods to initialize the data members:

```
setRollNo(int rollNo)
setName(String name)
setMarks(double marks)
```

Also, create a method named displayDetails() to display the student's details.

In the main() method:

Create a Student object.

Read the values from the user.

Initialize the data members using the setter methods only.

Call the displayDetails() method to display the student's information.

Input Format

The first line contains an integer representing the roll number.

The second line contains the student's name.

The third line contains a decimal number representing the student's marks.

Output Format

Display the details in the following format:

Roll No: <rollNo>

Name: <name>

Marks: <marks>

Your Code

```
1 import java.util.Scanner;
2 public class Main{
3     public static void main(String[] args){
4         Scanner s1=new Scanner(System.in);
5
6         int rollNo=s1.nextInt();
7         s1.nextLine();
8         String name=s1.nextLine();
9         double marks=s1.nextDouble();
10
11         Student s=new Student();
12
13         s.setRollNo(rollNo);
14         s.setName(name);
15         s.setMarks(marks);
16
17         s.displayDetails();
18     }
19 }
20 class Student{
21     private int rollNo;
22     private String name;
23     private double marks;
24
25     public void setRollNo(int rollNo){
26         this.rollNo=rollNo;
27     }
28     public void setName(String name){
29         this.name=name;
30     }
31     public void setMarks(double marks){
32         this.marks=marks;
33     }
34     public void displayDetails(){
35         System.out.println("Roll No: "+this.rollNo);
36         System.out.println("Name: "+this.name);
37         System.out.println("Marks: "+this.marks);
38     }
39 }
```

Input

```
20
Imraan
92
```

Output

```
Roll No: 20
Name: Imraan
Marks: 92.0
```

Visible Test Cases

Test Case 1	✓
Test Case 2	✓
Test Case 3	✓

10.00 / 10 marks

Q3 Student Details Using Default and Parameterized Constructors

Write a Java program to create a class named Student with the following data members:

int rollNo
String name
double marks

Implement the following constructors:

Default Constructor

Initializes the data members with the following values:

Roll Number: 101

Name: Rahul

Marks: 85.5

Parameterized Constructor

Accepts the roll number, name, and marks as parameters and initializes the data members.

Create a method named displayDetails() to display the student's details in the following format:

Roll No: <rollNo>

Name: <name>

Marks: <marks>

Task

In the main() method:

Create the first Student object using the default constructor and display its details.

Read the roll number, name, and marks from the user.

Create the second Student object using the parameterized constructor and display its details.

Input Format

The first line contains an integer representing the roll number.

The second line contains the student's name.

The third line contains a decimal number representing the student's marks.

Output Format

Display the details of both students.

The first student's details should be displayed using the default constructor, followed by the second student's details using the parameterized constructor.

Your Code

```
1 import java.util.Scanner;
2 public class main{
3     public static void main(String[] args){
4         Scanner s1=new Scanner(System.in);
5
6         Student s=new Student();
7         s.displayDetails();
8
9         int rollNo=s1.nextInt();
10        s1.nextLine();
11        String name=s1.nextLine();
12        double marks=s1.nextDouble();
13
14        Student s2=new Student(rollNo,name,marks);
15        s2.displayDetails();
16    }
17 }
18 class Student{
19     private int rollNo;
20     private String name;
21     private double marks;
22
23     public Student(){
24         this.rollNo=101;
25         this.name="Rahul";
26         this.marks=85.5;
27     }
28     public Student(int rollNo,String name,double marks){
29         this.rollNo=rollNo;
30         this.name=name;
31         this.marks=marks;
32     }
33     public void displayDetails(){
34         System.out.println("Roll No: "+ this.rollNo);
35         System.out.println("Name: "+ this.name);
36         System.out.println("Marks: "+ this.marks);
37     }
38 }
```

Input

```
21
Imraan
92
```

Output

```
Roll No: 101
Name: Rahul
Marks: 85.5
Roll No: 21
Name: Imraan
Marks: 92.0
```

Visible Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks

Q4 Net Salary Calculation Using Single Inheritance

Write a Java program to demonstrate Single Inheritance.

Create a superclass named Employee with the following data members:

```
int empId
String empName
double basicSalary
```

Create a method named getEmployeeDetails(int empId, String empName, double basicSalary) to initialize the employee details.

Create a subclass named Salary that inherits from Employee.

In the subclass, create a method named calculateSalary() that calculates the following:

House Rent Allowance (HRA) = 20% of Basic Salary

Dearness Allowance (DA) = 10% of Basic Salary

Net Salary = Basic Salary + HRA + DA

Create another method named displayDetails() to display the employee details and the calculated salary.

Input Format

The first line contains an integer representing the Employee ID.

The second line contains the Employee Name.

The third line contains the Basic Salary.

Output Format

Display the output in the following format:

Employee ID: <empId>

Employee Name: <empName>

Basic Salary: <basicSalary>

HRA: <hra>

DA: <da>

Net Salary: <netSalary>

Your Code

```
1 import java.util.Scanner;
2 public class Main{
3     public static void main(String[] args){
4         Scanner s1=new Scanner(System.in);
5
6         int empId=s1.nextInt();
7         s1.nextLine();
8         String empName=s1.nextLine();
9         double basicSalary=s1.nextDouble();
10
11         Salary s=new Salary();
12         s.getEmployeeDetails(empId,empName,basicSalary);
13         s.calculateSalary();
14         s.displayDetails();
15     }
16 }
17 class Employee{
18     int empId;
19     String empName;
20     double basicSalary;
21
22     public void getEmployeeDetails(int empId,String empName,double empId){
23         this.empId=empId;
24         this.empName=empName;
25         this.basicSalary=empbasicSalary;
26     }
27 }
28 class Salary extends Employee{
29     double hra,da,netSalary;
30
31     public void calculateSalary(){
32         this.hra=this.basicSalary*0.20;
33         this.da=this.basicSalary*0.10;
34         this.netSalary=this.basicSalary+this.hra+this.da;
35     }
36     public void displayDetails(){
37         System.out.println("Employee ID: "+this.empId);
38         System.out.println("Employee Name: "+this.empName);
39         System.out.println("Basic Salary: "+this.basicSalary);
40         System.out.println("HRA: "+this.hra);
41         System.out.println("DA: "+this.da);
42         System.out.println("Net Salary: "+this.netSalary);
43     }
44 }
```

Input

```
101
Imraan
50000
```

Output

```
Employee ID: 101
Employee Name: Imraan
Basic Salary: 50000.0
HRA: 10000.0
DA: 5000.0
Net Salary: 65000.0
```

Visible Test Cases

- Test Case 1 ✓
- Test Case 2 ✓
- Test Case 3 ✓

10.00 / 10 marks

Q5 Employee Gross Salary Calculation Using Multilevel Inheritance

Write a Java program to demonstrate Multilevel Inheritance.

Create three classes as described below:

Class 1: Employee (Superclass)

Data Members:

int empId

String empName

Method:

getEmployeeDetails(int empId, String empName) – Initializes the employee details.

Class 2: Salary (Derived Class)

This class inherits from Employee.

Data Member:

double basicSalary

Method:

getBasicSalary(double basicSalary) – Initializes the basic salary.

Class 3: GrossSalary (Derived Class)

This class inherits from Salary.

Method:

calculateSalary() – Calculates:

House Rent Allowance (HRA) = 20% of Basic Salary

Dearness Allowance (DA) = 10% of Basic Salary

Gross Salary = Basic Salary + HRA + DA

Method:

displayDetails() – Displays the employee details and the calculated salary.

Input Format

The first line contains an integer representing the Employee ID.

The second line contains the Employee Name.

The third line contains the Basic Salary.

Output Format

Display the output in the following format:

Employee ID: <empId>

Employee Name: <empName>

Basic Salary: <basicSalary>

HRA: <hra>

DA: <da>

Gross Salary: <grossSalary>

```
1 import java.util.Scanner;
2 public class Main{
3     public static void main(String[] args) {
4         Scanner S1=new Scanner(System.in);
5
6         int empId=S1.nextInt();
7         S1.nextLine();
8         String empName=S1.nextLine();
9         double basicSalary=S1.nextDouble();
10
11         GrossSalary g= new GrossSalary();
12
13         g.getEmployeeDetails(empId,empName);
14         g.getBasicSalary(basicSalary);
15         g.calculateSalary();
16         g.displayDetails();
17     }
18 }
19 class Employee{
20     int empId;
21     String empName;
22
23     public void getEmployeeDetails(int empId,String empName){
24         this.empId=empId;
25         this.empName=empName;
26     }
27 }
28 class Salary extends Employee{
29     double basicSalary;
30
31     public void getBasicSalary(double basicSalary){
32         this.basicSalary=basicSalary;
33     }
34 }
35 class GrossSalary extends Salary{
36     double hra, da,grossSalary;
37
38     public void calculateSalary(){
39         this.hra=this.basicSalary*0.20;
40         this.da=this.basicSalary*0.10;
41         this.grossSalary=this.basicSalary+this.hra+this.da;
42     }
43
44     public void displayDetails(){
45         System.out.println("Employee ID: "+this.empId);
46         System.out.println("Employee Name: "+this.empName);
47         System.out.println("Basic Salary: "+this.basicSalary);
48         System.out.println("HRA: "+this.hra);
49         System.out.println("DA: "+this.da);
50         System.out.println("Gross Salary: "+this.grossSalary);
51     }
52 }
```

101
Kaif
50000

Output

```
Employee ID: 101
Employee Name: Kaif
Basic Salary: 50000.0
HRA: 10000.0
DA: 5000.0
Gross Salary: 65000.0
```

Visible Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks